

第十一讲：深度学习与图像分类



授课人：王国泰 wgt_cv@163.com

电子科技大学机械与电气工程学院

2026-4-7



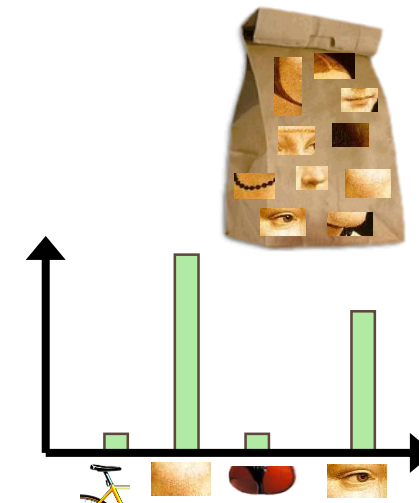
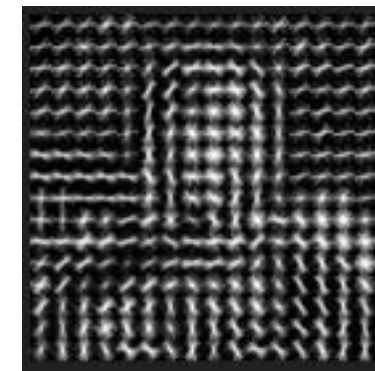
知识回顾：传统视觉方法



Best performing visions systems have commonality:

Hand designed features 人工提取特征

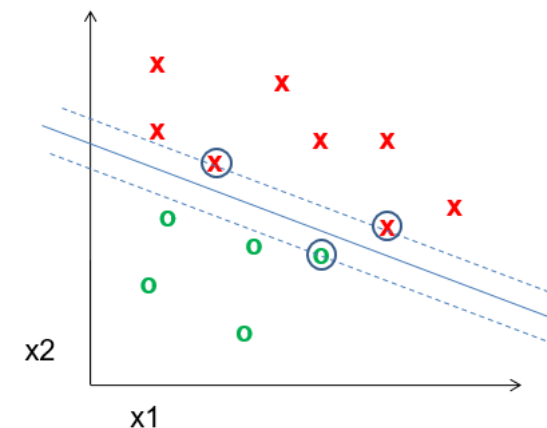
- Gradients + non-linear operations (exponentiation, clamping, binning)
- Features in combination (parts-based models)
- Multi-scale representations



Machine learning from databases 机器学习模型

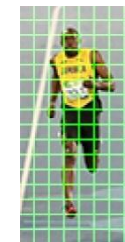
Linear classifiers (SVM)

- Some non-linear kernel tricks

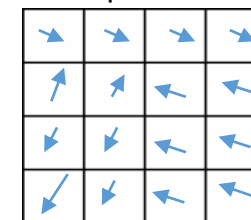


知识回顾：传统的目标检测方法

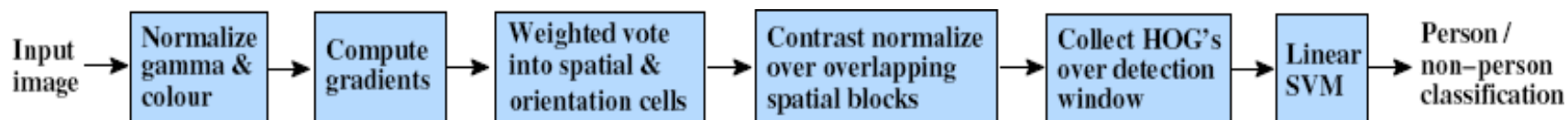
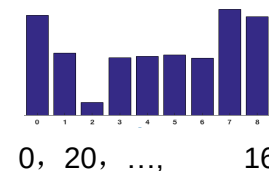
- Dalal-Triggs 行人检测



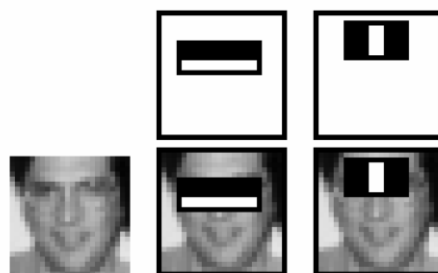
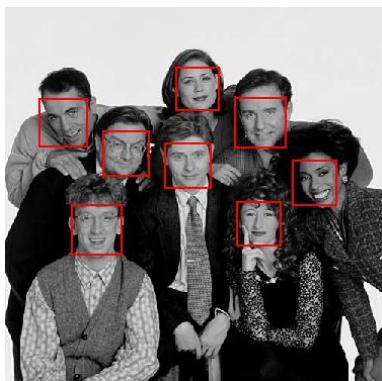
Histograms over $k \times k$ pixel cells



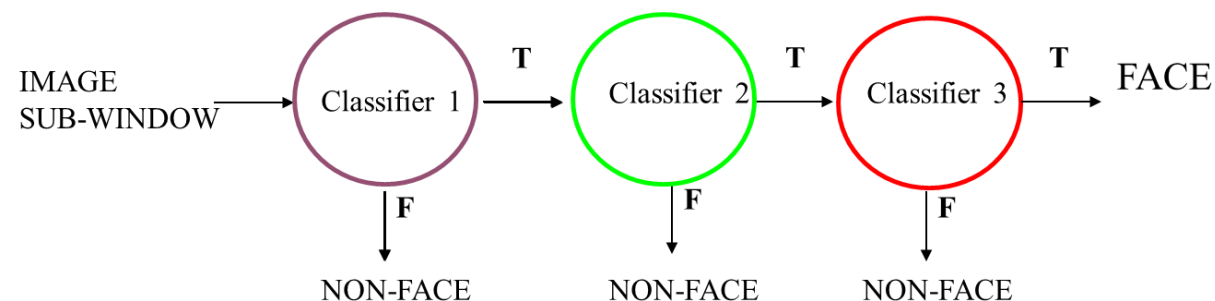
histogram



- Viola-Jones 人脸检测



Haar-like features



The pipeline we have learned so far



But it's still not that good...

PASCAL VOC = ~75% 20-class

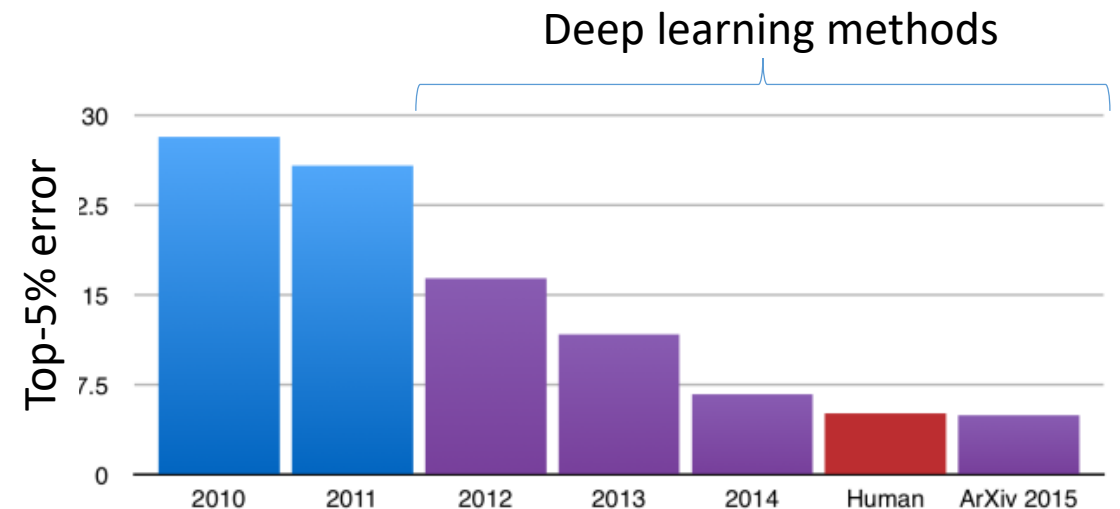
ImageNet = ~75% 1000-class, top 5

ImageNet (human) = ~95%

Deep learning can outperform human

Problems:

- Lossy features
- Lossy quantization
- 'Imperfect' classifier



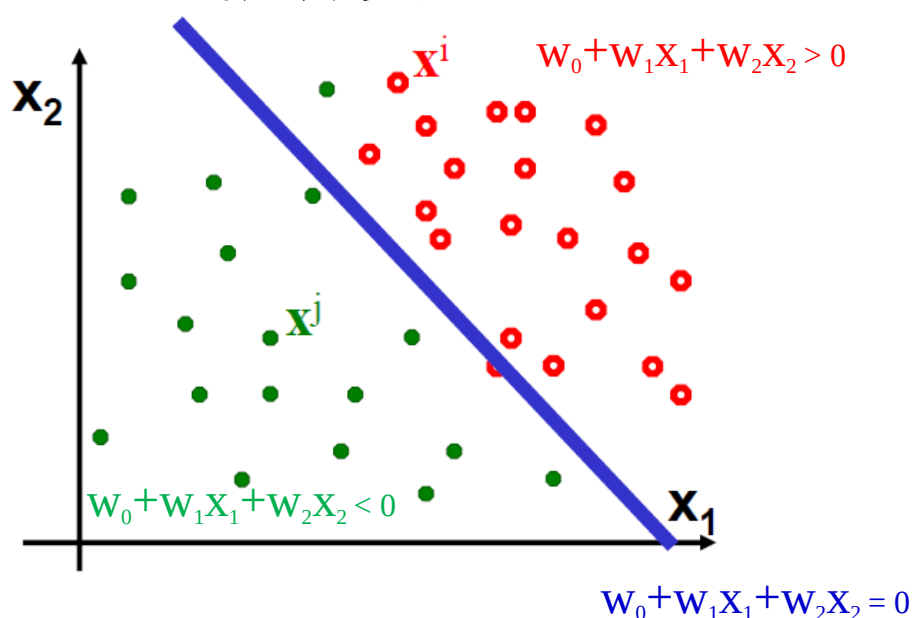
目录



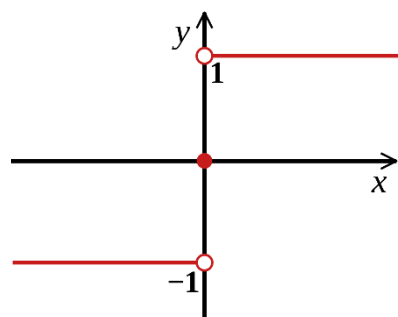
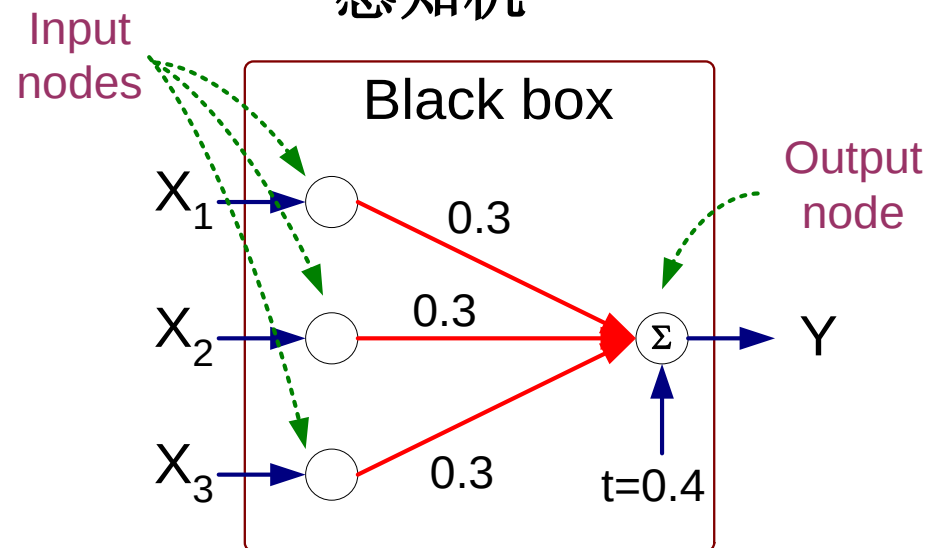
- 深度学习简介
- 卷积神经网络

Perceptron Model (感知机模型)

线性分类器

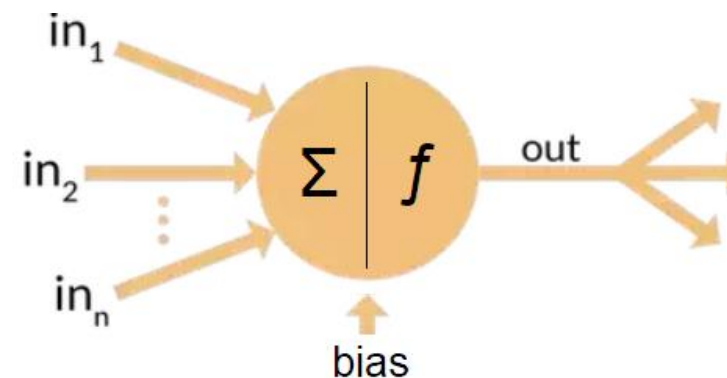


感知机

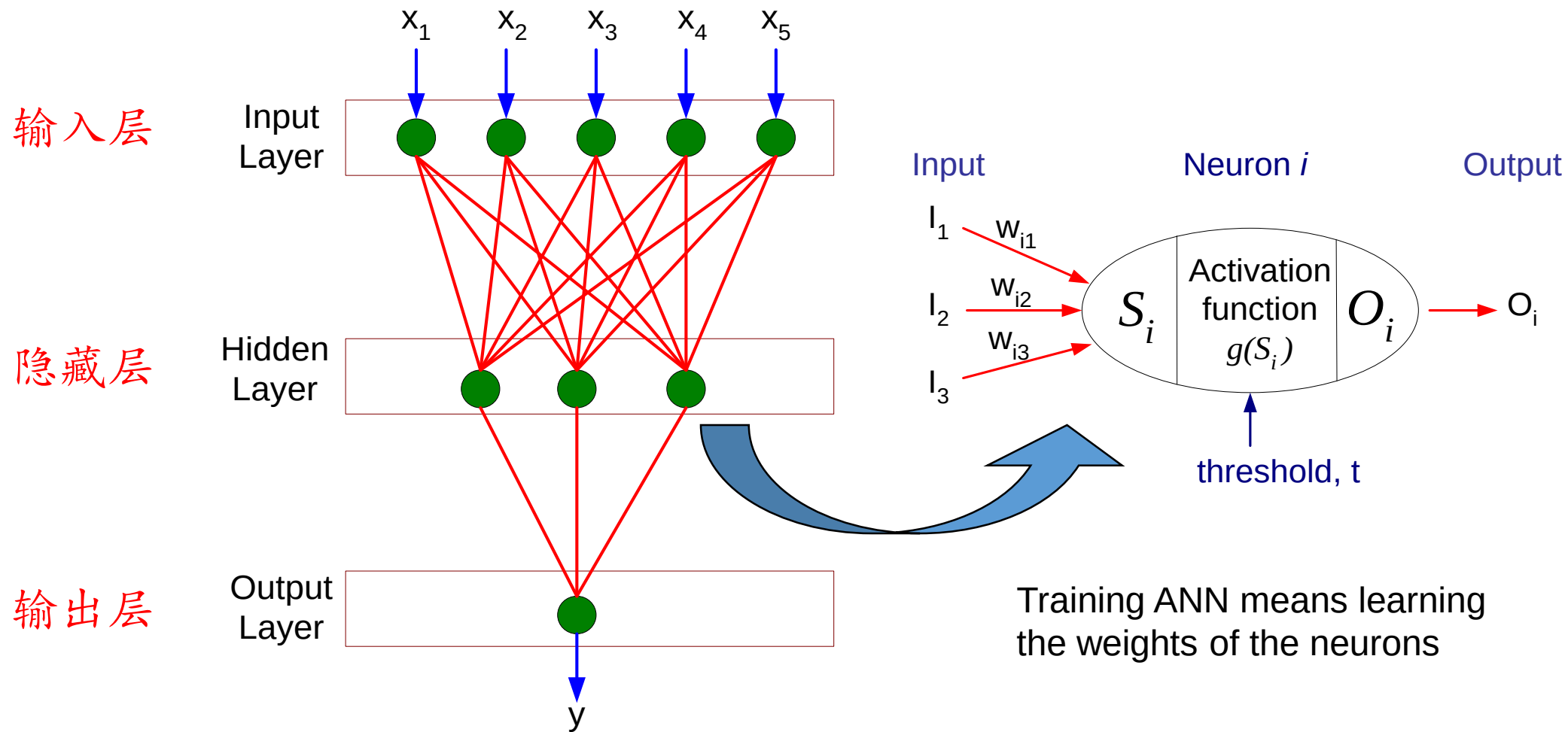


$$Y = \text{sign}(0.3X_1 + 0.3X_2 + 0.3X_3 - 0.4)$$

$$\text{where } \text{sign}(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ -1 & \text{if } x < 0 \end{cases}$$

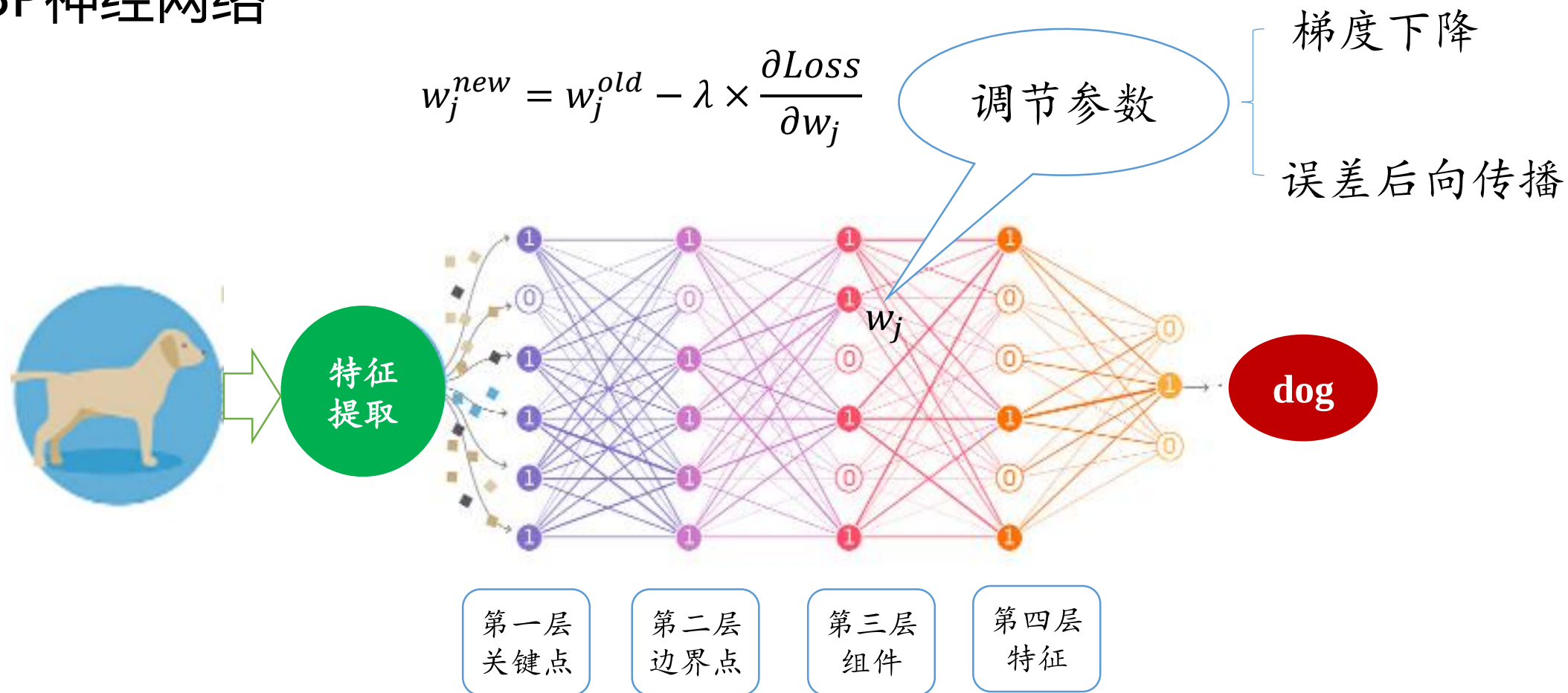


Artificial Neural Network 人工神经网络



Learning Multi-layer Neural Network

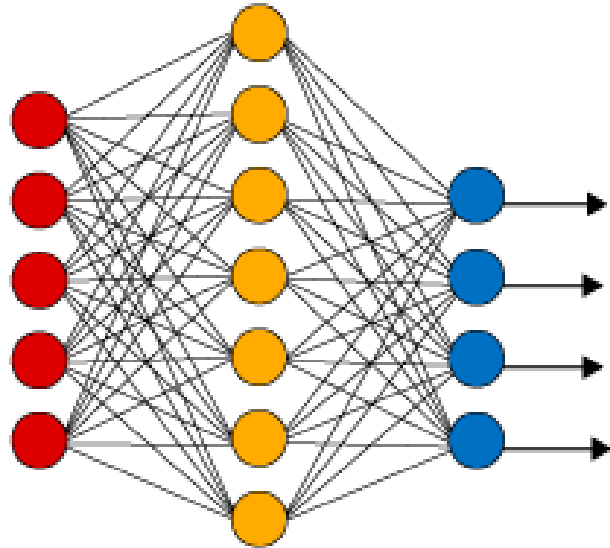
- BP神经网络



What is Deep Learning (深度学习)



Simple Neural Network

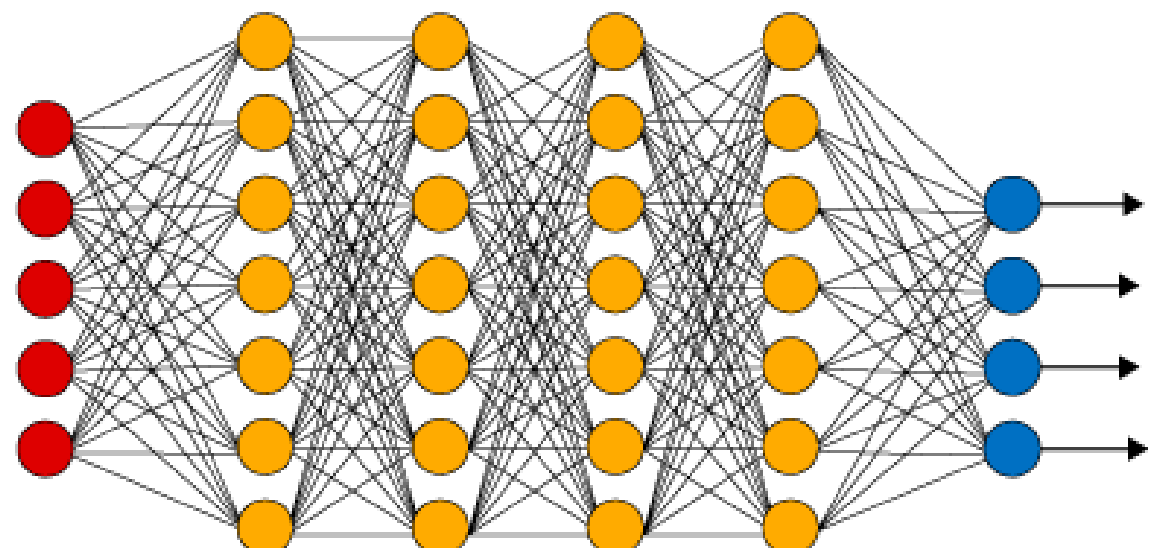


● Input Layer

● Hidden Layer

● Output Layer

Deep Learning Neural Network



Deep Neural Networks (DNN)

- 深度学习三位奠基人

- Geoffrey Hinton, Yann LeCun, 和Yoshua Bengio共同获得了2018年的图灵奖



Hinton: 1986年发明反向传播
“Learning Internal
Representations by Error
Propagation”

LeCun: 1980年代发明了卷积神经网络，现在已经成为了机器学习领域的基础技术之一。首次将卷积神经网络用于手写数字识别

Bengio: 1990年代发明的Probabilistic models of sequences。把神经网络和概率模型（例如隐马尔可夫模型）结合在一起

• 2024年诺贝尔物理学奖

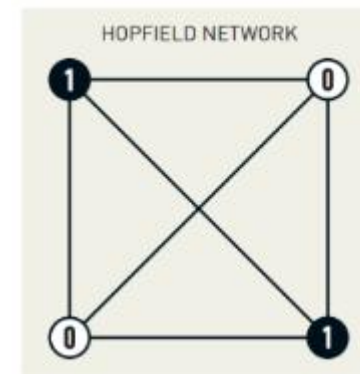
2024年诺贝尔物理学奖授予“在人工神经网络机器学习方面的基础性发现和发明”（for foundational discoveries and inventions that enable machine learning with artificial neural networks）



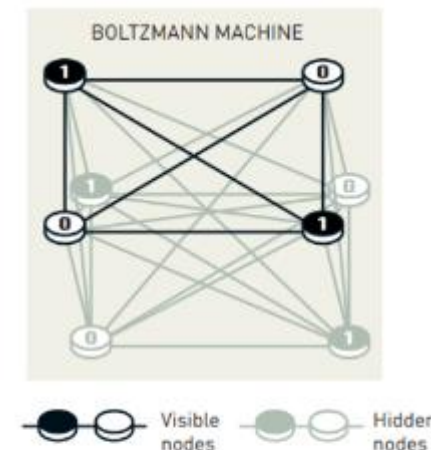
Ill. Niklas Elmehed © Nobel Prize Outreach
John J. Hopfield



Ill. Niklas Elmehed © Nobel Prize Outreach
Geoffrey E. Hinton



John Hopfield's associative memory is built so that all the nodes are connected to each other. Information is fed in and read out from all the nodes.

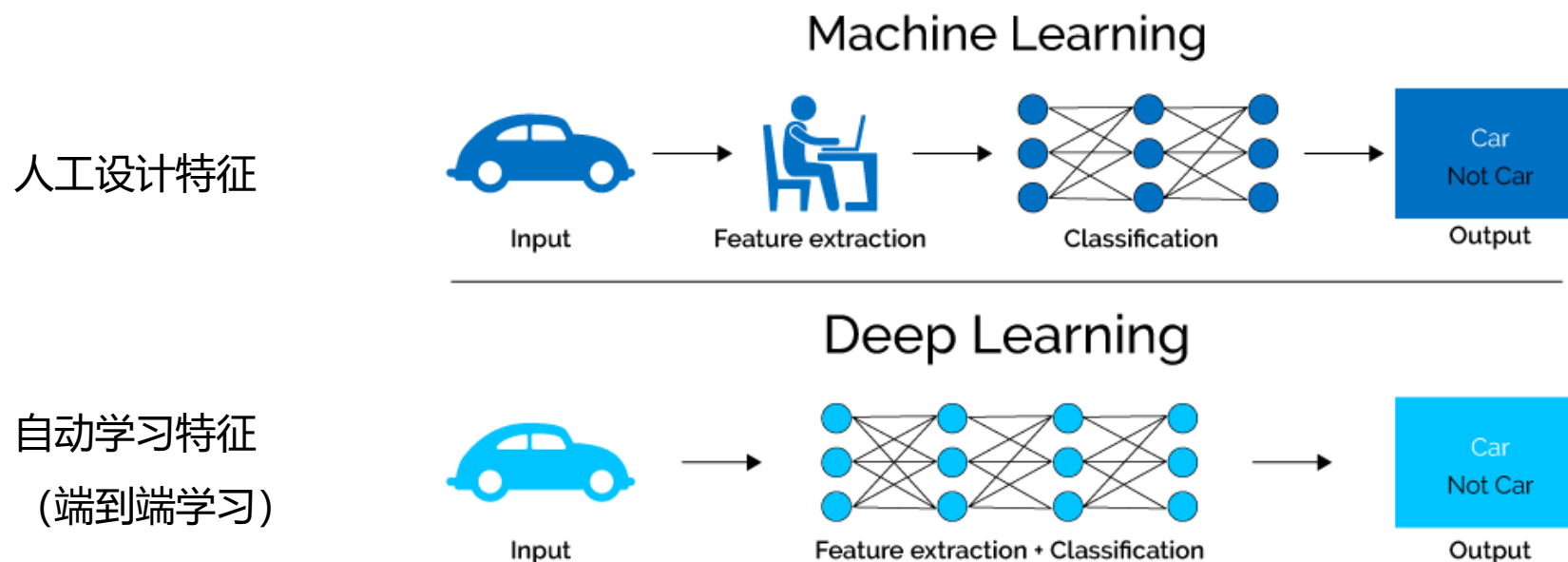


Geoffrey Hinton's Boltzmann machine is often constructed in two layers, where information is fed in and read out using a layer of visible nodes. They are connected to hidden nodes, which affect how the network functions in its entirety.

深度学习与传统机器学习方法的区别

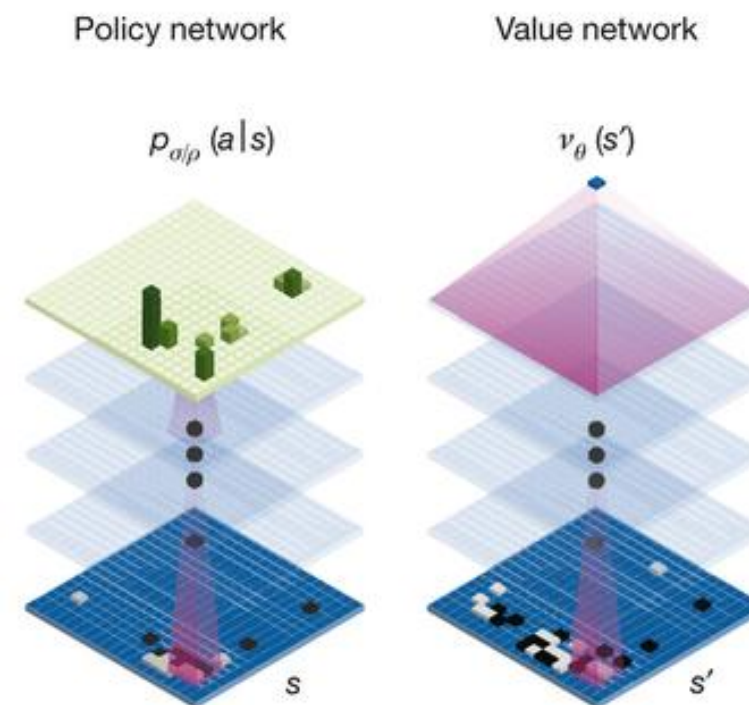
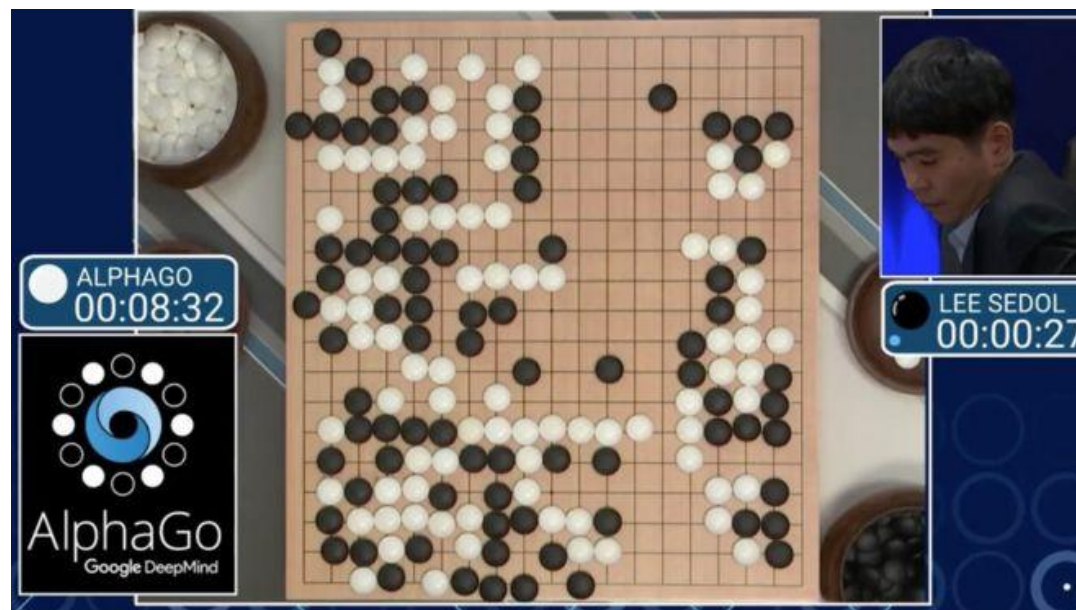


- Traditional machine learning vs deep learning
 - Traditional machine learning extract features first, then use classifier
 - Deep learning attempts to learn (multiple levels of) representation by using a **hierarchy of multiple layers, i.e., automatic feature learning**



深度学习的成功案例

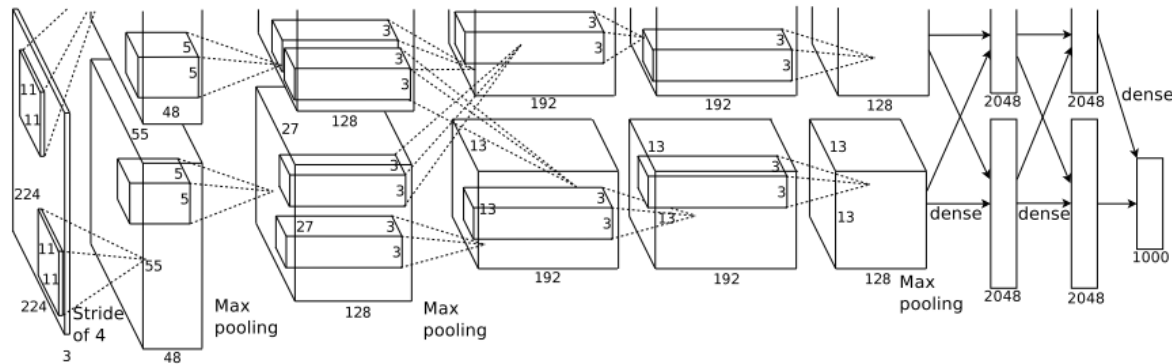
- Deep CNN in AlphaGO



- Policy network and value network
 - Deep networks to predict the probability of actions of next step
 - Learned from 160,000 games played human experts

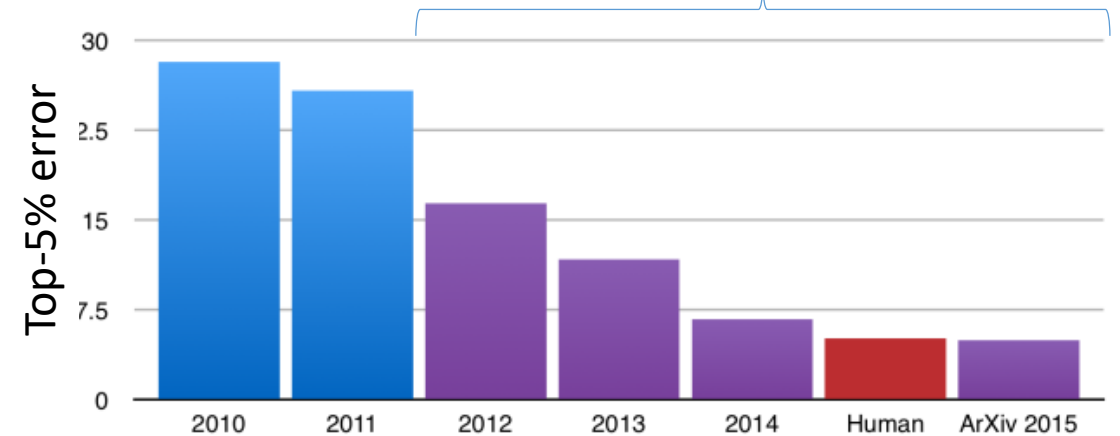
深度学习的成功案例

- AlexNet
 - Winner of ImageNet Large Scale Visual Recognition Challenge 2012
 - Learned from 15 million labeled images in over 1,000 categories
 - Took 5-6 days for training with 2 GPUs



Cited **>120000** times according to Google Scholar

Deep learning methods



深度学习的成功案例

- Reasons for success

Large amount
of data



15 million images

Computational
equipment

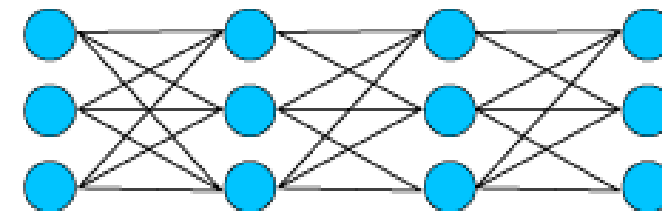


GPU (NVIDIA)



TPU (Google)

Deep network
(integration of feature
extraction)



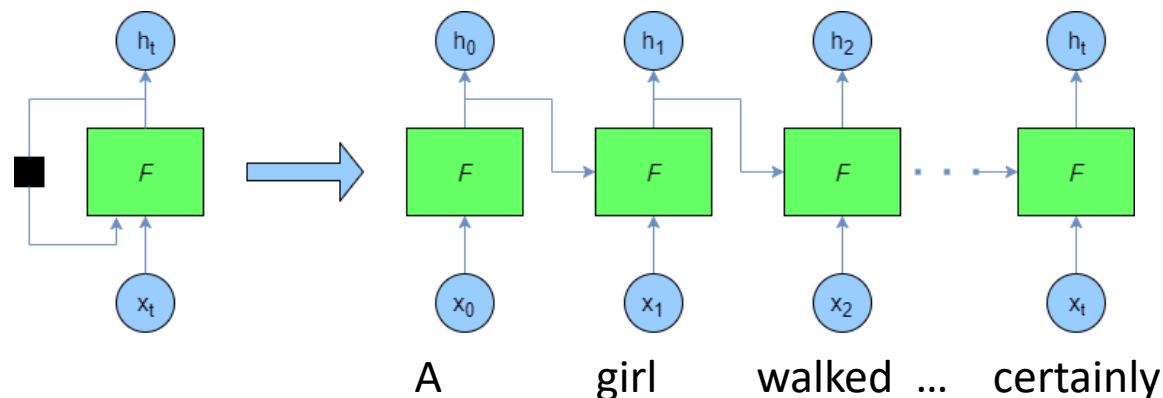
Feature extraction + Classification

深度神经网络

- Recurrent Neural Network

- Data can flow in any direction
- Language modeling

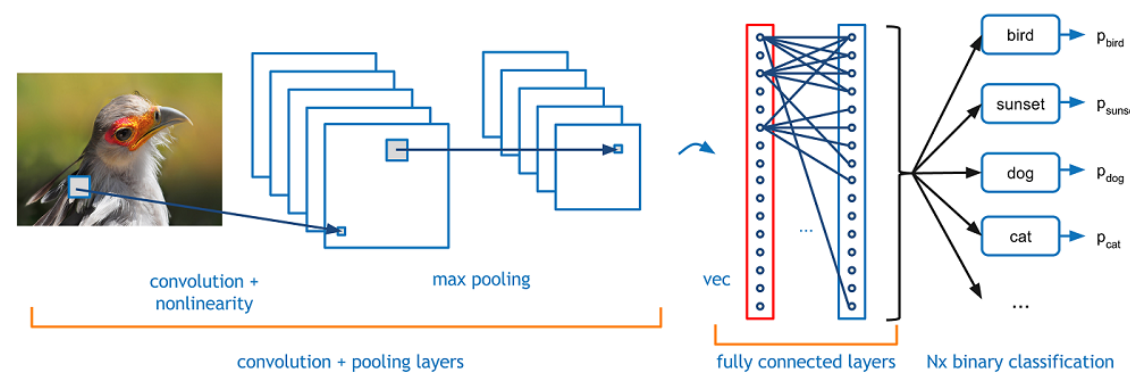
循环神经网络



- Convolutional Neural Network

- Mostly used for computer vision
- Can also be used for speech recognition

卷积神经网络

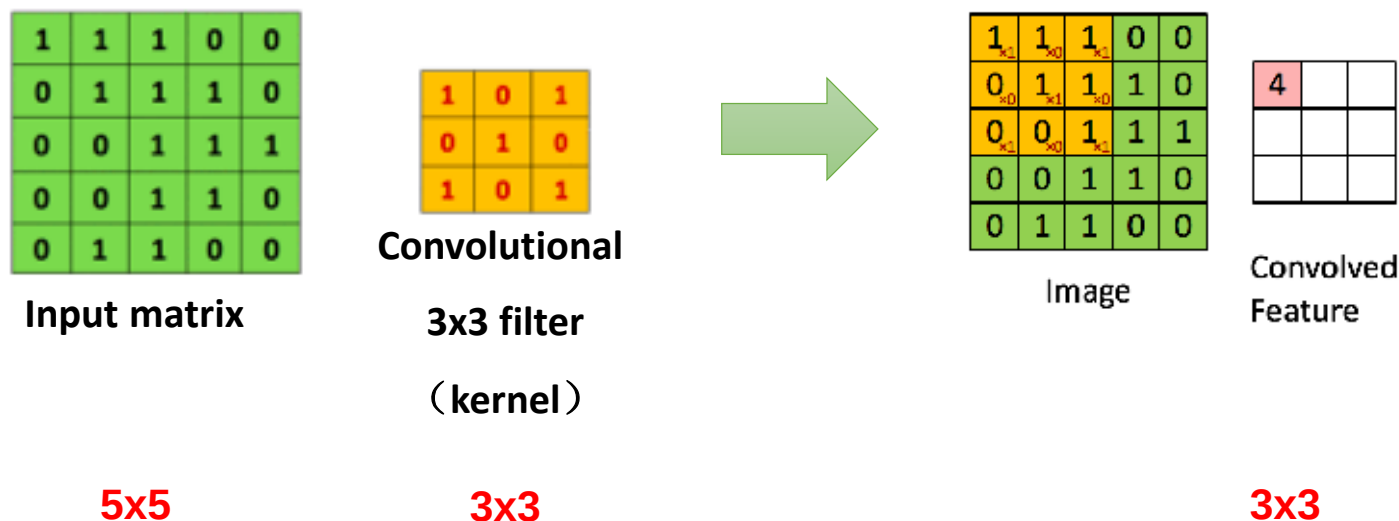


目录



- 深度学习简介
- 卷积神经网络
 - 卷积层
 - 池化层
 - 激活函数
 - Softmax与损失函数

- 卷积层 (Convolution layer)
 - 通过卷积核来提取特征
 - 特点：共享权重、位移不变性
 - 参数：卷积核的大小（如3x3），步长（如1x1）



- 卷积的例子

$$h[m, n] = \sum_{u=-k}^k \sum_{v=-k}^k f[u, v] \cdot I[m + u, n + v]$$

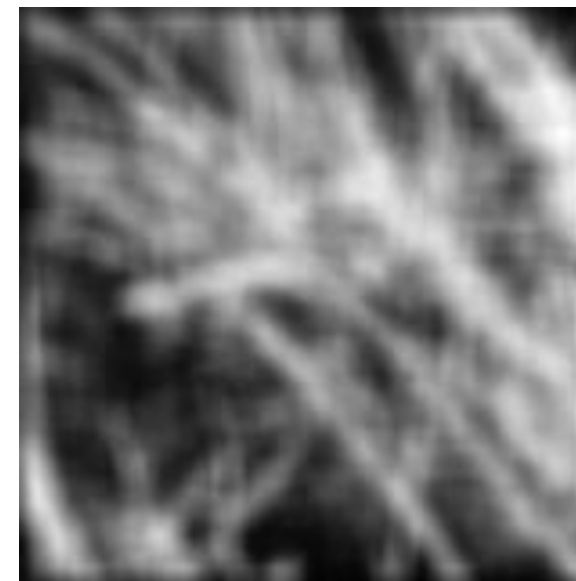
注：这里的卷积和相关性计算未做区分，标准的卷积定义使用 $m-u, n-v$



$$\frac{1}{9} f[\cdot, \cdot]$$

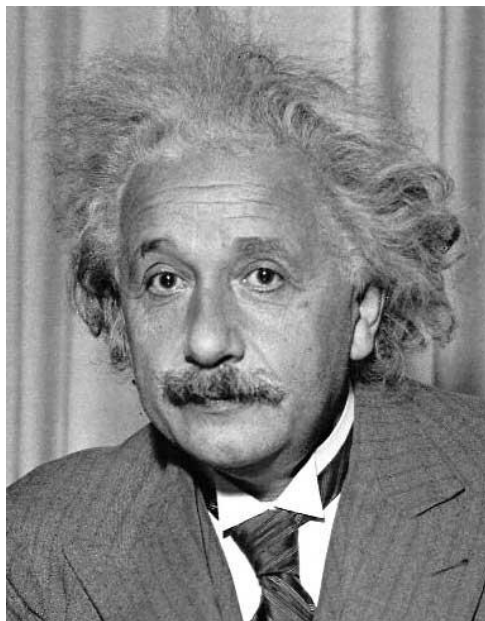
1	1	1
1	1	1
1	1	1

均值滤波



- 卷积的例子

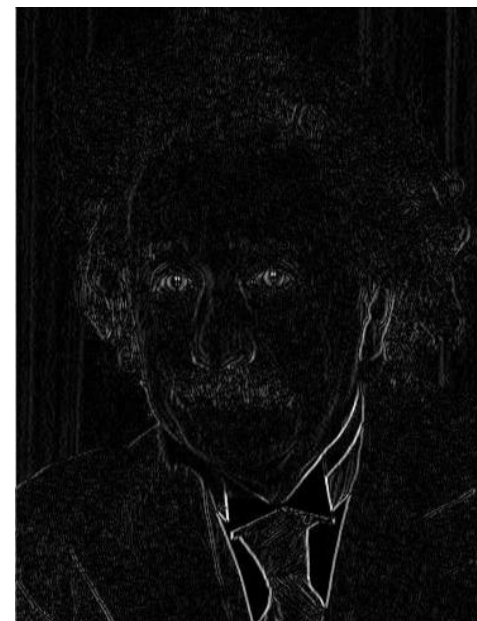
$$h[m, n] = \sum_{u=-k}^k \sum_{v=-k}^k f[u, v] \cdot I[m + u, n + v]$$



$f[\cdot, \cdot]$

1	0	-1
2	0	-2
1	0	-1

边缘检测
(Sobel算子)

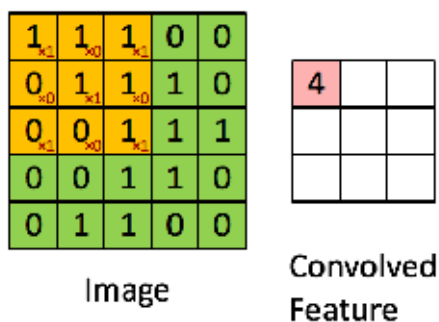


卷积神经网络

- 卷积层：填充

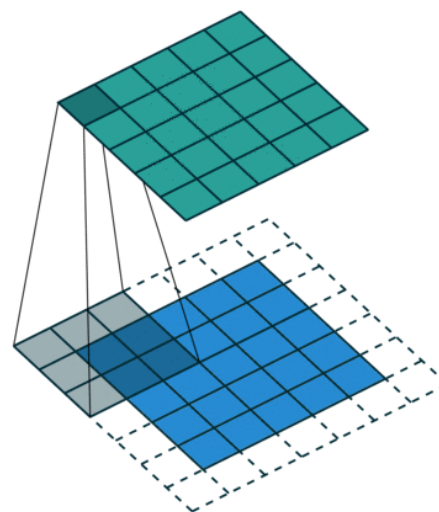
- 假设输入大小是5x5, 卷积核大小3x3, 步长为1

$$W' = (W - F + 2P) / S + 1$$



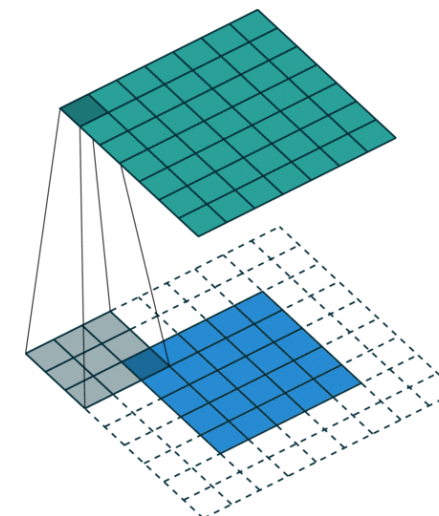
Valid 模式 (p = 0)
输出大小: 3x3

$$W' = (5 - 3) / 1 + 1$$



Same 模式 (p = 1)
输出大小: 5x5

$$W' = (5 - 3 + 2) / 1 + 1$$



full 模式 (p = 2)
输出大小: 7x7

$$W' = (5 - 3 + 4) / 1 + 1$$

卷积神经网络



- 卷积层：填充
 - 如何设置填充区域的值？

0	0	0	0	0	0	0
0	1	2	6	4	9	0
0	3	5	7	9	6	0
0	4	3	4	5	1	0
0	4	8	7	9	5	0
0	5	1	6	6	5	0
0	0	0	0	0	0	0

Zero padding

3	3	5	1	1
3	3	5	1	1
3	3	6	1	1
4	4	7	9	9
4	4	7	9	9

Reflective padding

6	3	6	1	6
5	3	5	1	5
6	3	6	1	6
7	4	7	9	7
6	3	6	1	6

Replication padding

卷积神经网络

- 卷积层：步长 (stride)

$$W' = (W - F + 2P) / S + 1$$

$$W = 5$$

$$F = 3$$

$$P = 1$$

$$S = 2$$

$$W' = (5 - 3 + 2) / 2 + 1 = 3$$

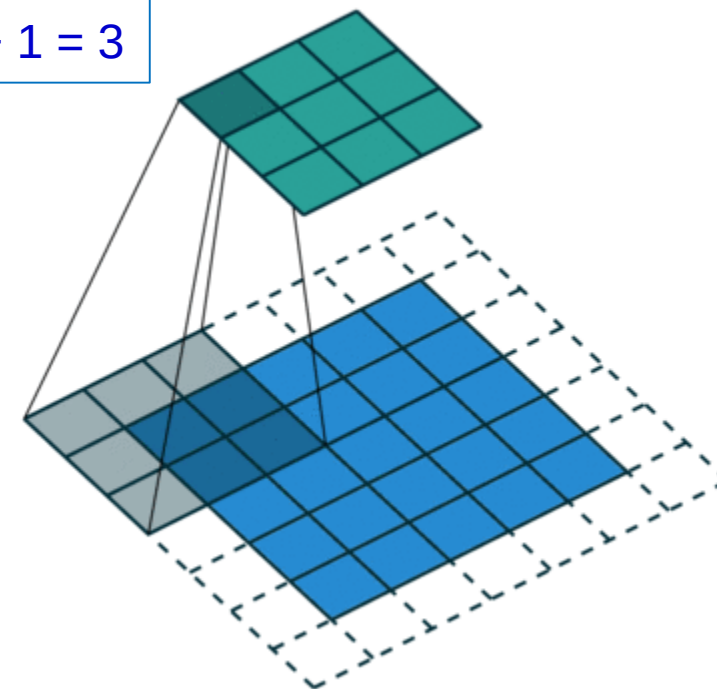
0	0	0	0	0	0	0
0	1	2	1	1	1	0
0	1	1	5	3	9	0
0	2	4	4	7	5	0
0	3	6	7	5	6	0
0	1	6	5	3	1	0
0	0	0	0	0	0	0

5x5 Image



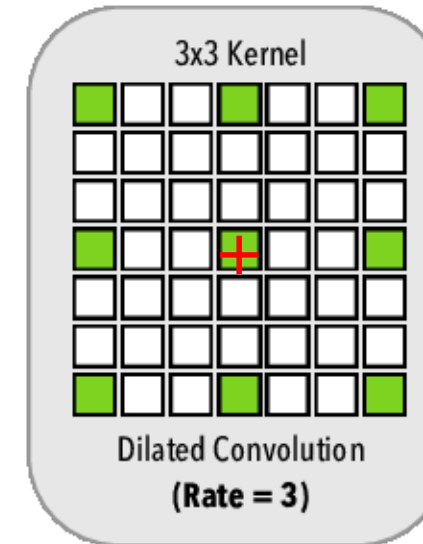
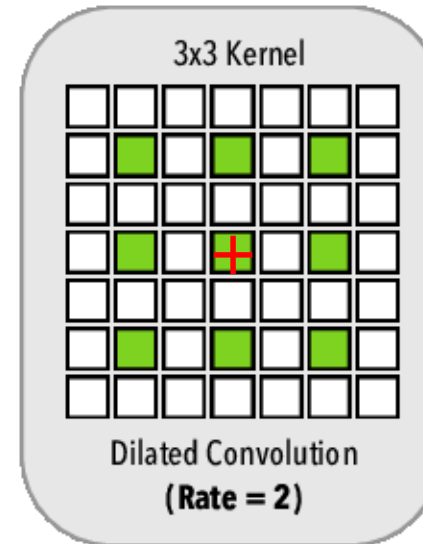
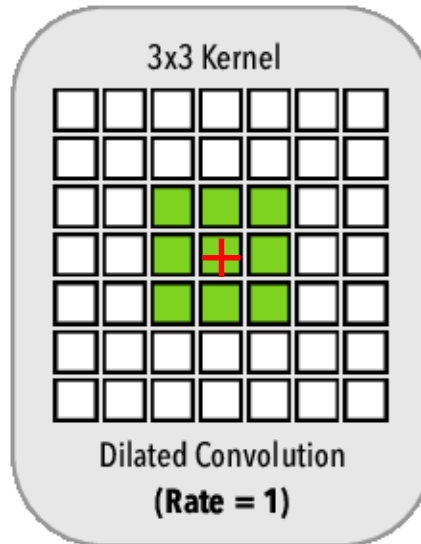
5	13	14
17	42	35
16	32	15

3x3



Padding = 1, stride = 2

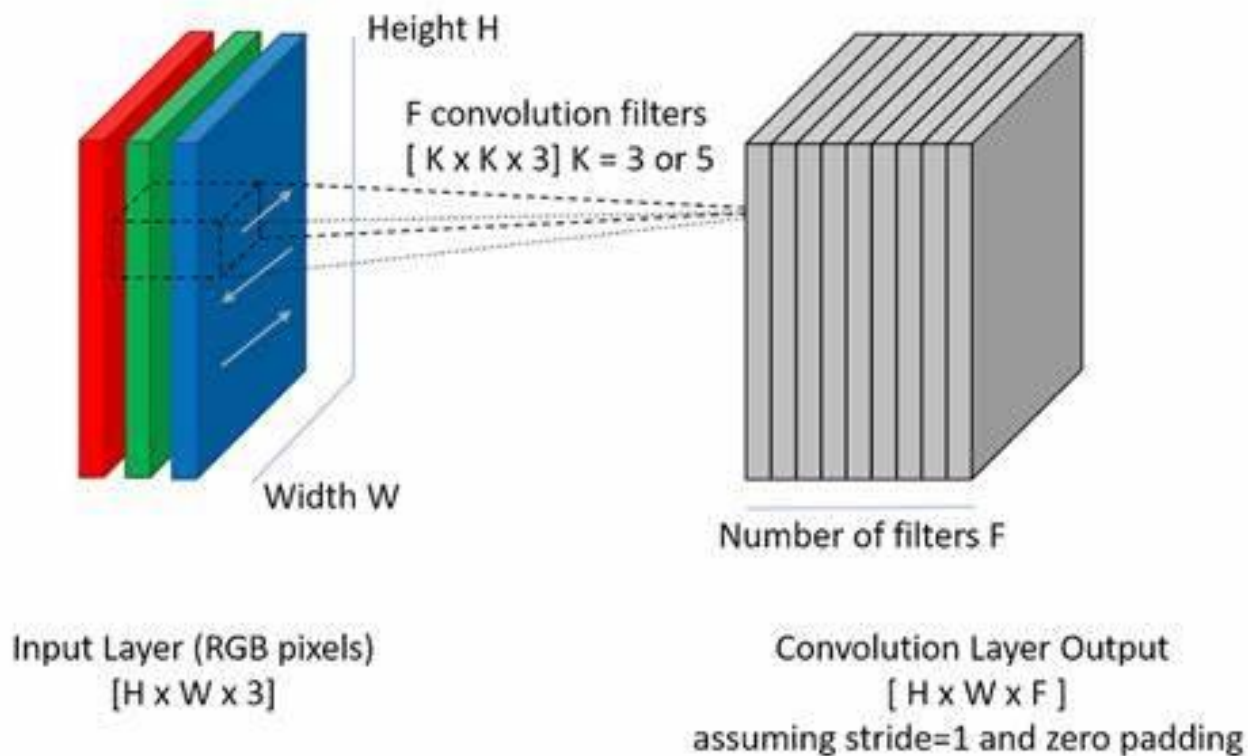
- 卷积层：空洞卷积 (dilated convolution)
- 将卷积核的元素向外扩张，等效于空洞填充，空洞参数为 r
- $r=1$, 相当于标准卷积
- $r>1$, 可在参数量一样的情况下增加感受野



- 卷积层：多通道输出
- 如果输入为三通道图像，需要输出F个特征图
- 需要F个 $K \times K \times 3$ 的卷积核

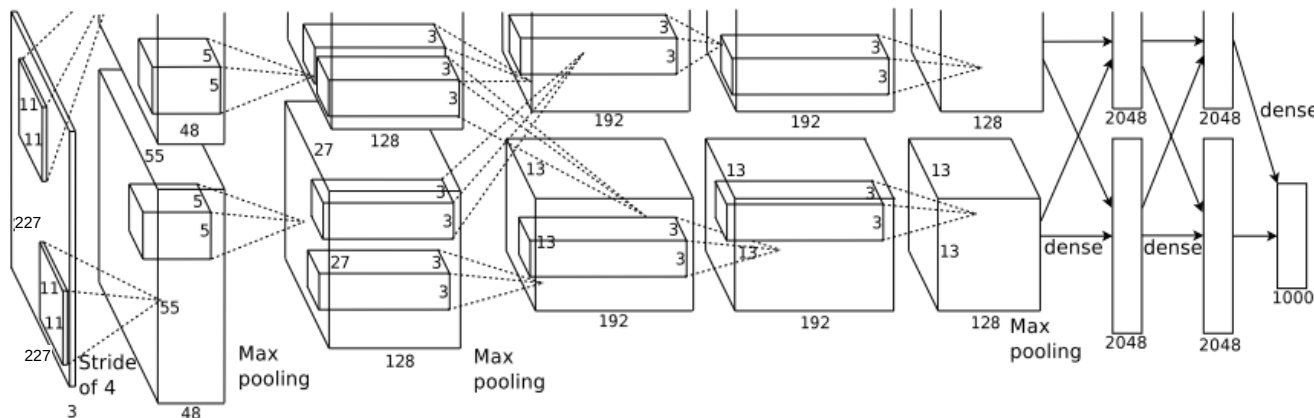
整个卷积层的参数有: $K \times K \times 3 \times F$

一般地，如果输入和输出通道分别是 C_{in} 和 C_{out} , 卷积层的权重参数数量为 $K \times K \times C_{in} \times C_{out}$

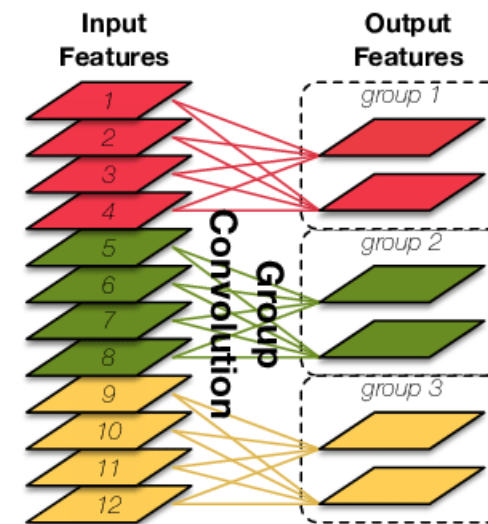
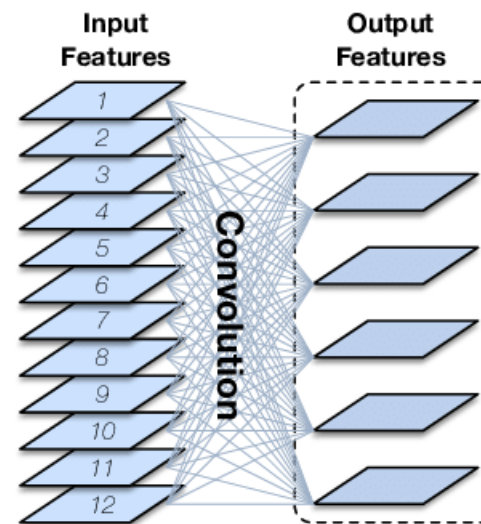


卷积神经网络

- 卷积层：分组卷积 (grouped convolution)
- 将输入的特征通道分为G个组
- 每一个组分别进行卷积操作，结果拼接起来
- 参数量减少G倍



AlexNet 将特征通道分为2 组.
每一组分别放在一个GPU上, 由于当时GPU容量限制



- 卷积层：分组卷积 (grouped convolution)
- 特殊情况：深度可分离卷积 (depthwise separable convolution)
- 分组数 = 输入通道数

通常与 1×1 卷积结合使用，常用在轻量级CNN中

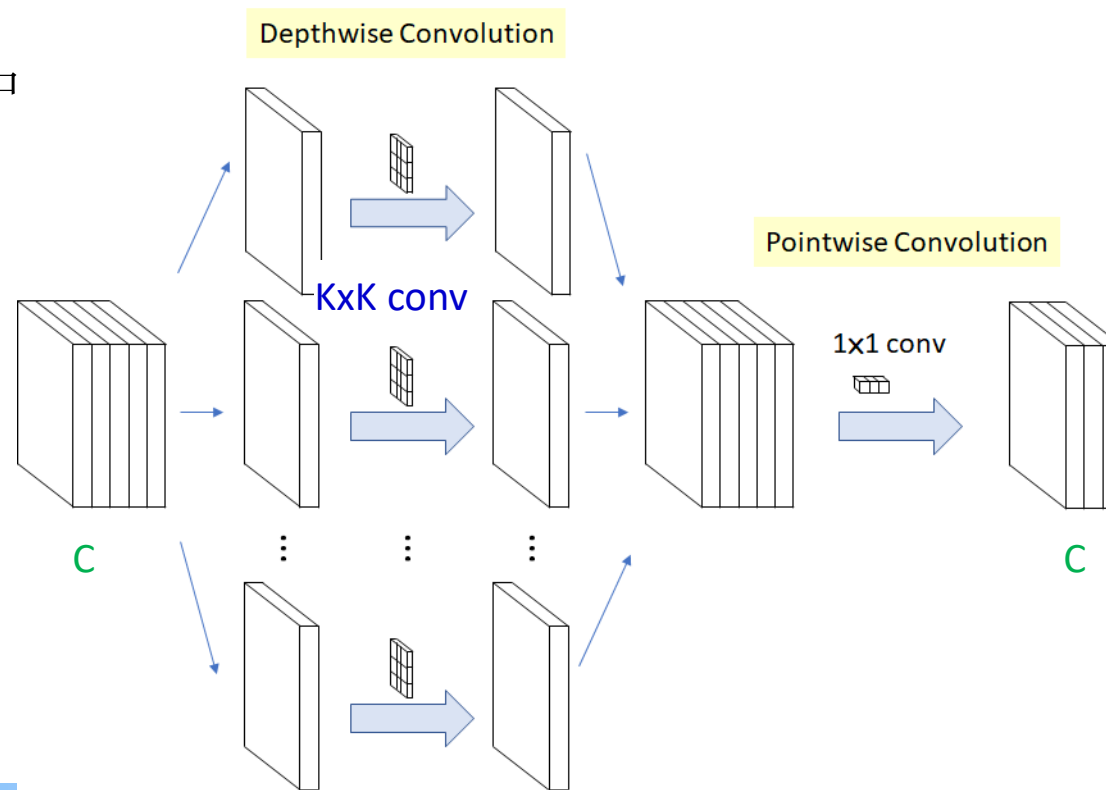
假设输入和输出通道均为 C , 卷积核大小为 $K\times K$

- 常规卷积的参数量为

$$K\times K\times C\times C$$

- 使用深度可分离卷积和 1×1 卷积的参数量

$$K\times K\times C + C\times C$$



- 卷积层的参数总结如下

- 输入通道数、输出通道数
- 卷积核大小
- 填充大小
- 步长
- 空洞率
- 分组数

Pytorch
代码

Tensorflow
代码

CONV2D

```
CLASS torch.nn.Conv2d(in_channels: int, out_channels: int,  
kernel_size: Union[T, Tuple[T, T]], stride: Union[T, Tuple[T,  
T]] = 1, padding: Union[T, Tuple[T, T]] = 0, dilation:  
Union[T, Tuple[T, T]] = 1, groups: int = 1, bias: bool =  
True, padding_mode: str = 'zeros')
```

[SOURCE] [↗](#)

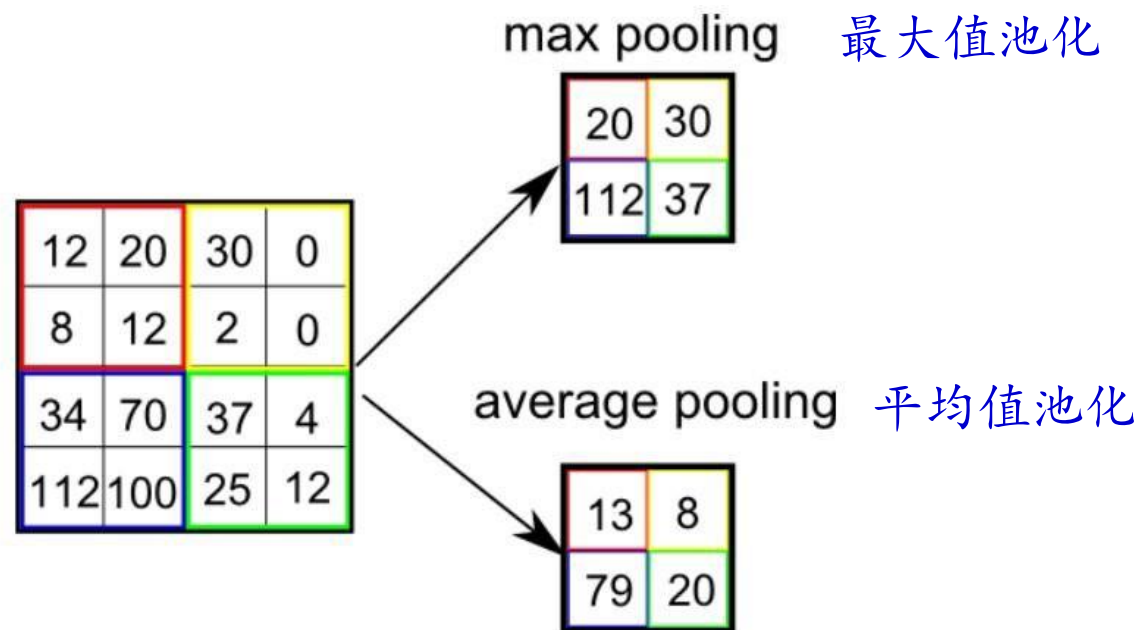
```
tf.keras.layers.Conv2D(  
    filters, kernel_size, strides=(1, 1), padding='valid',  
    data_format=None, dilation_rate=(1, 1), groups=1, activation=None,  
    use_bias=True, kernel_initializer='glorot_uniform',  
    bias_initializer='zeros', kernel_regularizer=None,  
    bias_regularizer=None, activity_regularizer=None, kernel_constraint=None,  
    bias_constraint=None, **kwargs  
)
```

• 池化层 (pooling layer)

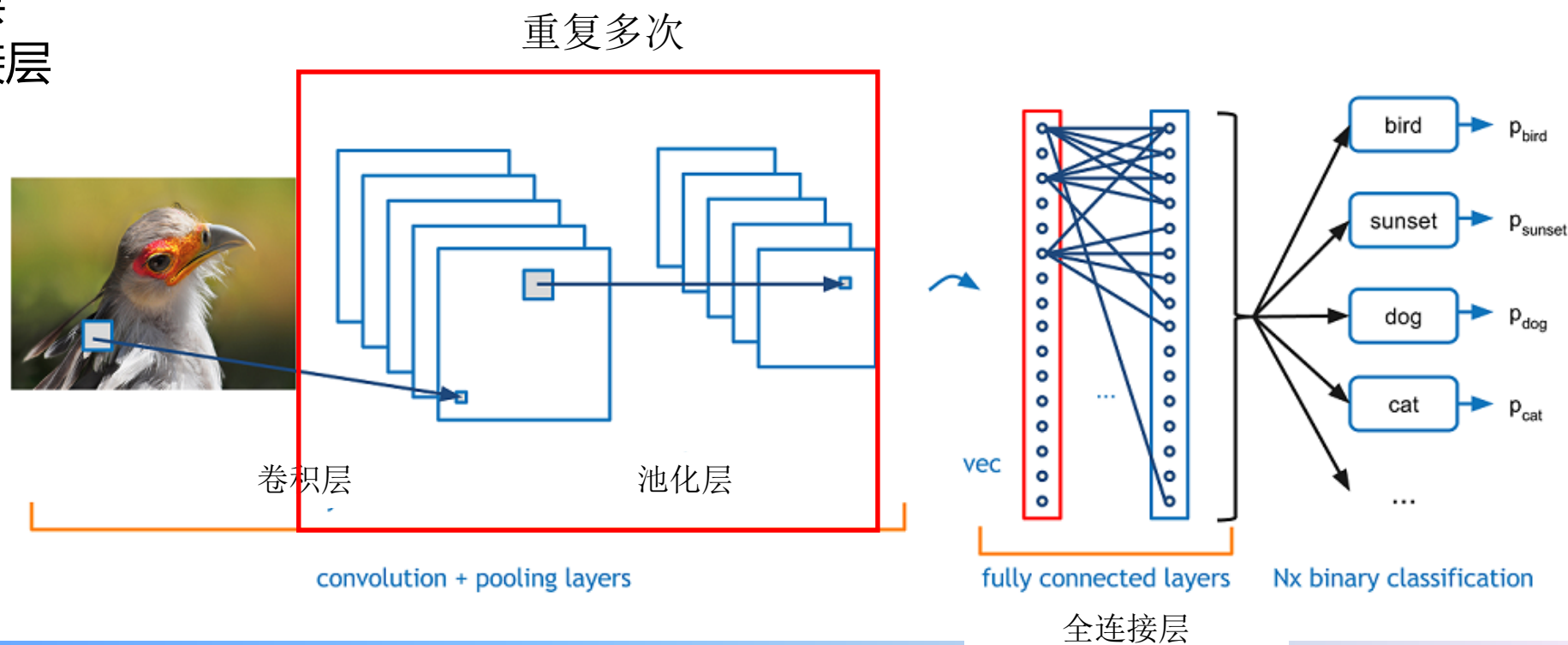
- 输入特征图通过池化层来降低分辨率
- 参数: 核的大小 (如 2×2), 步长 (如 2×2)

池化层仅改变空间大小, 不改变特征通道的数量

池化层中没有需要学习的参数



- CNN的基本结构
- 由以下层组合而成：
 - 卷积层（卷积后使用激活函数）
 - 池化层
 - 全连接层



目录



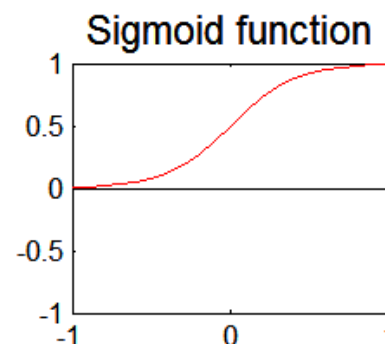
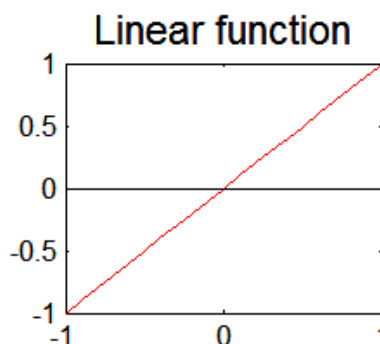
- 深度学习简介
- 卷积神经网络
 - 卷积层
 - 池化层
 - 激活函数
 - Softmax与损失函数

Various Activation Functions

- **Activation function** is usually denoted as $\sigma(\cdot)$ 激活函数

- Role: obtain non-linear mapping
- Can take different forms

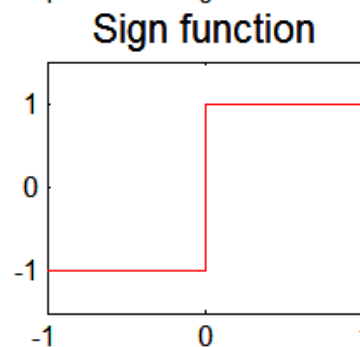
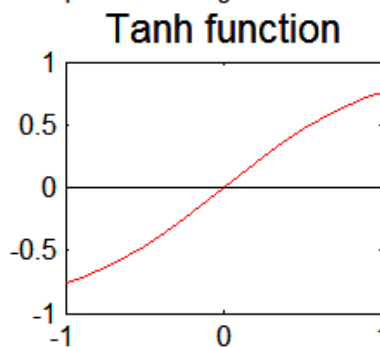
$$y = x$$



$$y = \frac{1}{1 + e^{-x}}$$

Tanh:
hyperbolic
tangent

$$y = \frac{e^{2x} - 1}{e^{2x} + 1}$$

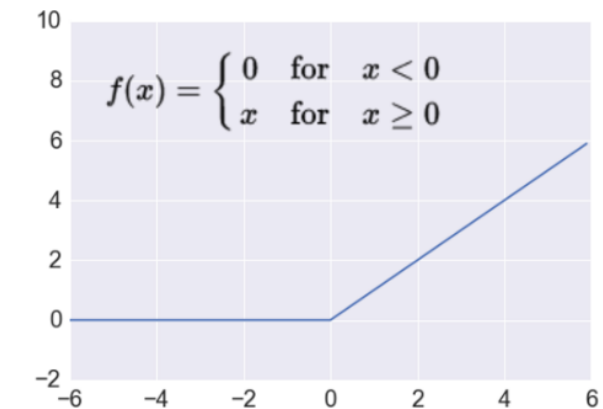
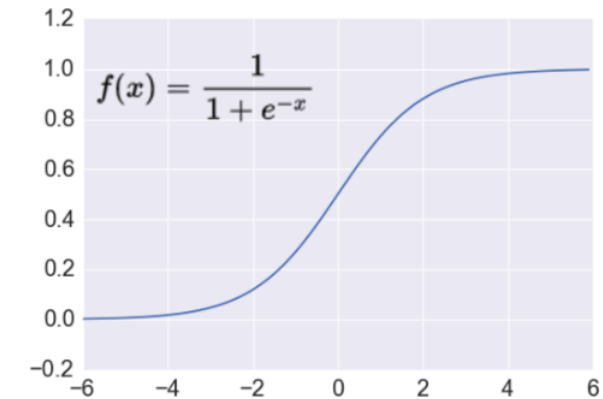


$$\text{sgn}(x) := \begin{cases} -1 & \text{if } x < 0, \\ 0 & \text{if } x = 0, \\ 1 & \text{if } x > 0. \end{cases}$$

Activations for DNN



- **Sigmoid**: not preferred for deep networks
 - Saturate (when activation is 0 or 1)
 - Gradient is close to 0 (**gradient vanish 梯度消失**)
 - Very slow to update weights
- **ReLU: Rectified Linear Unit**
 - Training much faster
 - Accelerates the convergence of SGD
 - Due to linear, non-saturating form
 - Less expensive for computing
 - Prevents the gradient vanishing problem



Dying ReLU problem: ReLU neurons can sometimes be pushed into states in which they become inactive for essentially all inputs.

Activations for DNN



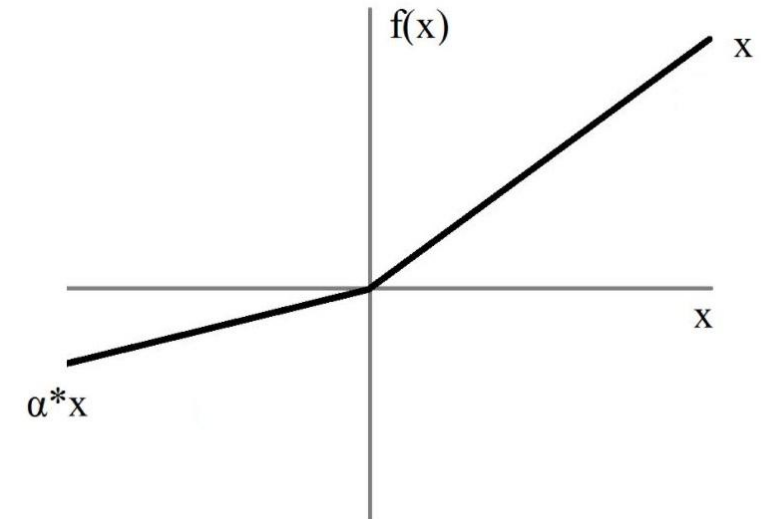
- Leaky ReLU

- allow a small, positive gradient when the unit is not active

$$f(x) = \begin{cases} x & \text{if } x > 0, \\ 0.01x & \text{otherwise} \end{cases}$$

- Parametric ReLU

- making the coefficient of leakage into a parameter that is learned along with the other neural-network parameters



$$f(x) = \begin{cases} x & \text{if } x > 0, \\ ax & \text{otherwise.} \end{cases}$$

PRELU

```
CLASS torch.nn.PReLU(num_parameters: int = 1, init: float = 0.25)
```

[SOURCE]

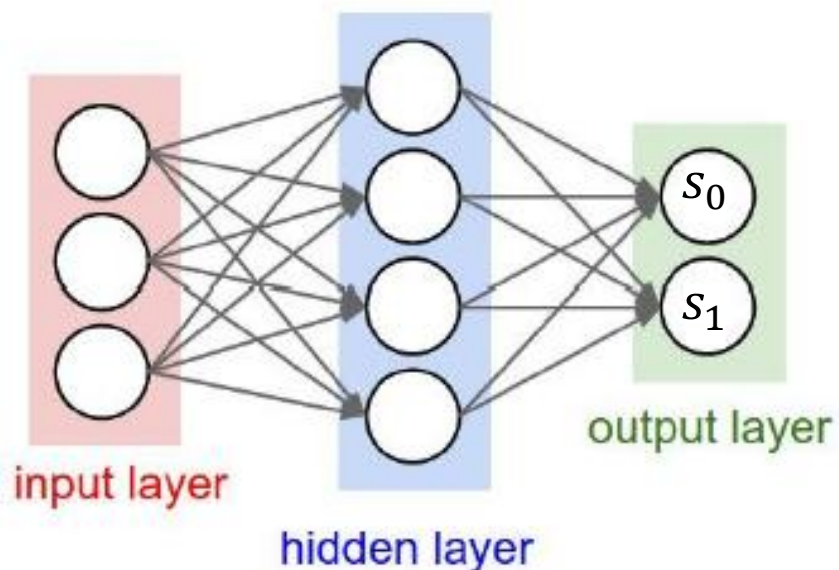
目录



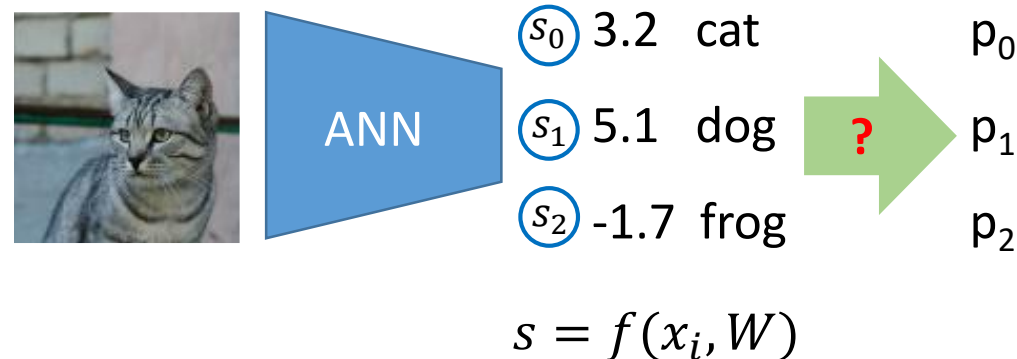
- 深度学习简介
- 卷积神经网络
 - 卷积层
 - 池化层
 - 激活函数
 - Softmax与损失函数

Softmax与损失函数

- 神经网络中每个输出节点得到一个与该类别相关的分数
- 如何将这些分数转换为**概率值**?



提示：每个值需要为正数，并且之和为1

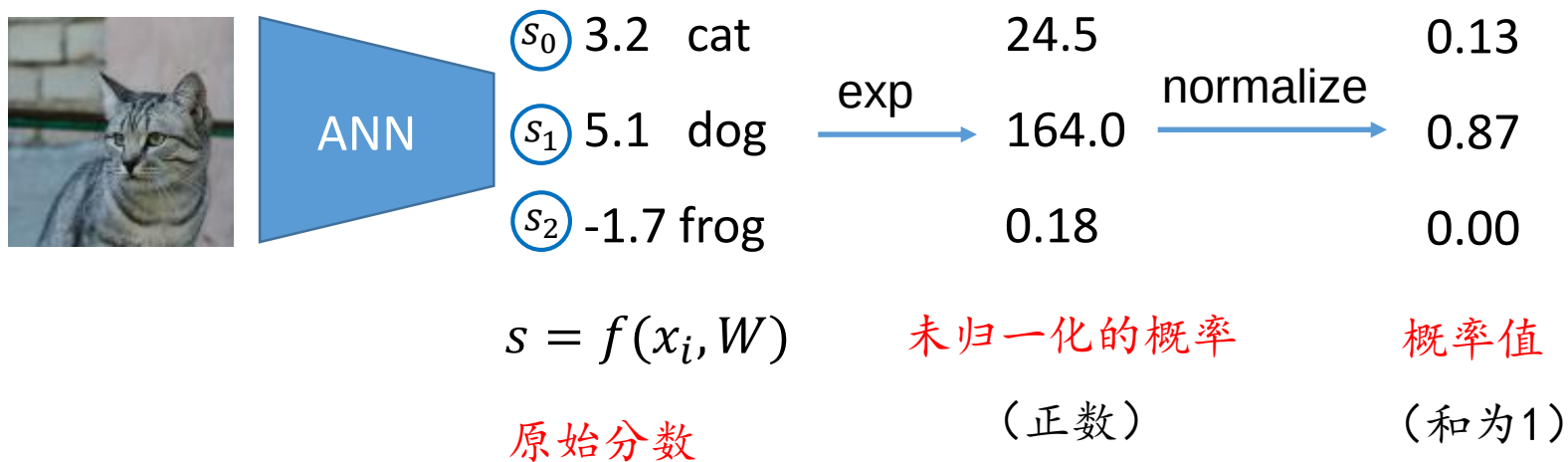


Softmax分类器

- 将输出节点的分数转换为**概率值**

Softmax

$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$



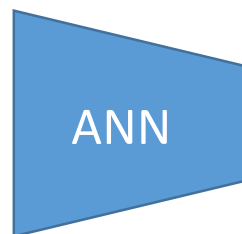
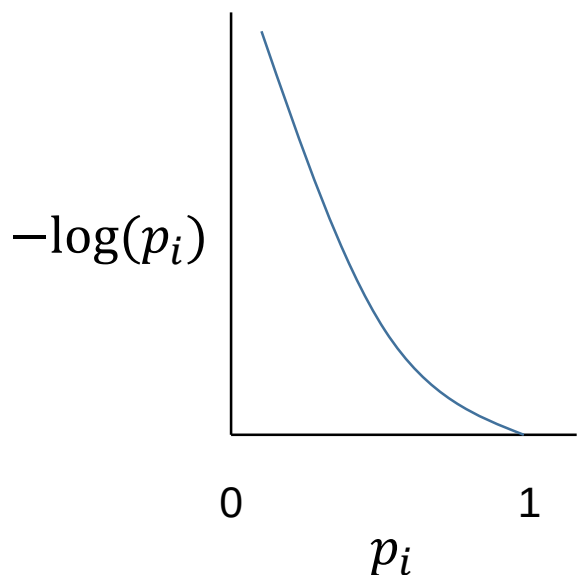
Softmax与损失函数



如何基于概率值定义损失函数？

- 提示：最大后验概率

$$L_i = -\log P(Y = y_i | X = x_i)$$



s_0 3.2 cat
 s_1 5.1 dog
 s_2 -1.7 frog

softmax

0.13
0.87
0.00

$y_0 = 1$
 $y_1 = 0$
 $y_2 = 0$

$s = f(x_i, W)$

原始分数

概率值
(和为1)

Ground truth

$$L_i = -\log(0.13) = -\sum_j y_j \log p_j = 2.04$$

交叉熵 (cross entropy)

Entropy

$$L_i = - \sum_{j=1}^K p_j \log p_j$$

Cross Entropy (log loss)

$$L_i = - \sum_{j=1}^K y_j \log p_j$$

对于二分类问题

$$\begin{aligned} L_i &= - \sum_{j=1}^2 y_j \log p_j = - y_1 \log p_1 - y_0 \log p_0 \\ &= - y \log p_1 - (1 - y) \log(1 - p_1) \end{aligned}$$

$y=0$ 或 1 是真实标签, p_1 是该样本属于类别1的概率

Softmax与损失函数



- Log loss = maximum likelihood estimation (最大似然估计)

$$\begin{aligned} & \operatorname{argmax}_{\theta} \prod_{i=1}^n q_{\theta}(y_i|x_i) \\ & \operatorname{argmax}_{\theta} \sum_{i=1}^n \log q_{\theta}(y_i|x_i) \\ & \operatorname{argmin}_{\theta} - \sum_{i=1}^n \log q_{\theta}(y_i|x_i) \end{aligned}$$

最大似然估计

$$\begin{aligned} p(y|x_i) &= \begin{cases} 1 & \text{if } y = y_i \\ 0 & \text{otherwise} \end{cases} \\ H_i(p, q_{\theta}) &= - \sum_{y \in Y} p(y|x_i) \log q_{\theta}(y|x_i) \\ &= - \log q_{\theta}(y_i|x_i) \end{aligned} \quad \text{交叉熵}$$
$$\begin{aligned} L &= \sum_{i=1}^n H_i(p, q_{\theta}) \\ &= \sum_{i=1}^n - \log q_{\theta}(y_i|x_i) \\ &= - \sum_{i=1}^n \log q_{\theta}(y_i|x_i) \end{aligned}$$

交叉熵损失函数

Softmax与损失函数



• Softmax 与Log损失联合使用?

- Softmax 通常用于分类网络的最后一层，得到各类别的概率（类别互斥）
- 预测的概率值输入交叉熵损失函数（log 损失）

